

Python

PROGRAMMING

(Beginner to Advanced)



This ebook, titled "Python Programming for Beginners", is the intellectual property of Mr_Circuits (Subrata Sasmal). All content, including but not limited to text, images, diagrams, code samples, and other materials, is protected under international copyright laws and treaties. The unauthorized reproduction, distribution, transmission, display, or modification of this work is strictly prohibited without the express written consent of the author.

Terms of Use

This ebook is provided for educational and informational purposes only. You may not modify, adapt, or reproduce any part of this ebook for commercial purposes without prior written approval from the author.

However, if you are a student, teacher, or individual engaged in self-learning, you are allowed to use the content of this ebook in the following ways:

You may download, print, or store a copy of this ebook solely for your personal, non-commercial use.

You may use the code examples and diagrams provided within this ebook in your personal projects, as long as proper credit is given to the author.

You are encouraged to share brief excerpts or quotations from this ebook, provided they are attributed to Mr_Circuits (Subrata Sasmal), and do not exceed a reasonable length or scope that would violate the integrity of the original work.

Prohibited Uses

Without the explicit permission of Mr_Circuits (Subrata Sasmal), the following actions are strictly prohibited:

Reproducing, transmitting, or distributing this ebook or its content, in whole or in part, through any means including but not limited to email, websites, social media platforms, or other digital communication methods.

Selling, renting, leasing, licensing, or otherwise offering this ebook for profit in any form.

Altering, editing, or creating derivative works based on the content of this ebook. Translating this ebook into other languages without prior written consent from the author.

Educational and Non-Commercial Use

If you are a teacher or educator wishing to use portions of this ebook for instructional purposes in a classroom, workshop, or similar setting, you may do so under the condition that the use is for non-commercial, educational purposes, and proper credit is given to the author. For more extensive use, including the reproduction of significant sections of the ebook, please contact the author for licensing and permission arrangements.

Attribution

If quoting or referencing sections of this ebook, please ensure proper attribution is given. Example attribution format:

"Python Programming for Beginners" by Mr_Circuits (Subrata Sasmal), © 2024. Reproduced with permission.

Contact Information

For permission requests, licensing inquiries, or other questions regarding the use of this ebook, please reach out to the author via email at: [insert email address or contact information here].

The author reserves the right to take legal action against any infringement of these terms, including but not limited to unauthorized reproduction, distribution, or commercial exploitation of this ebook.

Disclaimer

While the author has made every effort to ensure the accuracy and completeness of the information contained within this ebook, no warranties or guarantees are provided. The author assumes no responsibility for errors, omissions, or potential misinterpretation of the material. Readers are encouraged to apply their discretion and judgment when using the provided content.

- **In This Python Course, You Will Learn:**

Introduction to Python

- What is Python?
- Why choose Python for programming?
- Installing Python and setting up an IDE

Basic Syntax and Structure

- Understanding Python syntax
- Writing your first Python program (Hello World)
- Code structure: indentation and comments

Variables and Data Types

- What are variables?
- Python data types (integers, floats, strings, booleans)
- Type conversion and casting

Operators in Python

- Arithmetic operators
- Comparison and logical operators
- Assignment operators and their usage

Taking Input and Displaying Output

- Using `input()` for user input
- Printing output with the `print()` function
- String formatting (f-strings, `format()` method)

Control Flow in Python

- Using `if`, `elif`, and `else` statements for decision-making
- Looping with `for` and `while` loops
- Breaking out of loops with `break` and `continue`

Understanding Functions

- Defining and calling functions
- Function arguments and return values
- Local and global scope in functions

Working with Lists, Tuples, and Sets

- Creating and manipulating lists
- Understanding tuples and their immutability
- Sets: adding, removing, and checking elements

Mastering Dictionaries

- Creating and using dictionaries
- Accessing and modifying dictionary items
- Looping through dictionaries

File Handling

- Opening and reading files in Python
- Writing data to files
- Handling file-related exceptions

Exception Handling in Python

- Using try-except blocks to catch errors
- Handling specific exceptions
- Cleaning up with the finally block

Working with Python Modules and Libraries

- Importing and using Python's standard modules
- Introduction to useful libraries (math, random, datetime)
- Installing third-party libraries using pip

Introduction to Object-Oriented Programming (OOP)

- Creating classes and objects
 - Defining methods and attributes
 - Inheritance and polymorphism in Python
- etc.**

PYTHON PROGRAMMING

• Introducing Python-

- **Python is a high-level, general-purpose , and very popular Programming language.**
- **It is being used in web development, Machine learning applications , etc.**
- **Python programs are generally smaller than the other programming languages.**
- **Programmers → have to type relatively , makes them readable at the time .**
- **Very Simple Syntax. & Easy to learn.**

```
print("Hello")
```

➤ General purpose language (Simple yet very powerful language)

- **Console Application & Scripts**
- **Desktop Application**
- **Web Application**
- **Game Development**
- **Machine Learning, Deep learning, AI, Big data etc**

➤ Multiparadigm Support

- **Procedural Style Programming like C.**
- **OOPS like Java..**
- **Functional programming like Lisp.**

➤ Portable Or Platform Independent

Programs that are typically first compiled into intermediate code, then the code is run by the interpreter.

➤ Dynamically Typed

```
x = 10  
y = "geeks"  
x = "python"
```

This feature is different from C++ & Java. They are statically type . in python we don't have to specify the variable name.

But the in the more disadvantage is in this the run time of the program, so there are more chances of run time error.

➤ Automatic Garbage when Collection

It means when you dynamically allocate memory so you don't have to worry to realising this memory

➤ Popular Application Built in Python

Youtube, Netflix, Quora, Instagram, Dropbox

➤ print () in Python –

A some function is a set of instructions that take some input from us. do some work on those Parament and product some output .

. Example -

```
print ("Hello")  
print ("Welcome", "to", "GfG")  
print()  
print("Hope you are enjoying Python")
```

.Output- Hello

Welcome GFG

Hope you are enjoying Python

- and end and sep parameters in print()

**Eg- print ("Welcome", end = "")
print (" to GFG")**

Output - Welcome to GFG

End parameter creates space And it tells what should be printing after its printing. And in sep parameter when we have multiple input parameters we use sep to separate them. By default they are separated by space .

Eg- print ("10", "04", "2024", sep = "-")

Output- 10-04-2024

➤ Variables in Python -

Variable is a name location. Python variable is also known as an identifier and used to hold value.

In python we don't need to specify the type of variable because python is an interpreted language and smart enough to get variable type .

Example - price = 100

```
tax = 18
total-price price + tax
print (total-price)
```

Output - 118

How variables work?

```
x = 10
y = "geeks"
z = 20
```

if we say, $w = z$

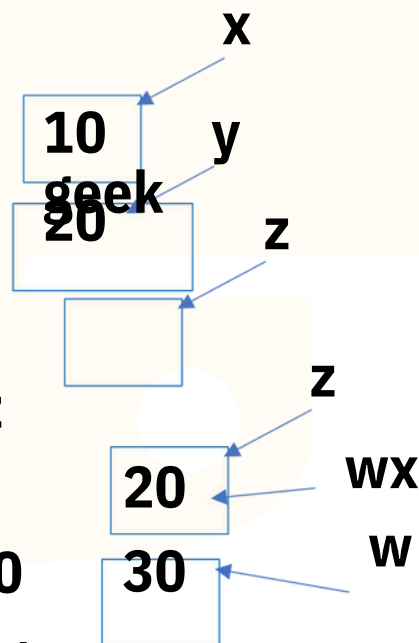
then

but if ,

$w = 30$

```
print (x)
print (y)
print (z)
print (w)
```

o/p - 10
geeks
20
30



➤ Python Is Dynamically Typed

X= 10

Print(X)

X= “geeks”

Print(x)

o/p - 10

geeks

Note- using a variable without assigning it causes error.

Eg – print (x)

o/p- name error : name is not assigned

➤ Input () in python –

This function is used to take input from the user

. Example – name = input (“ Enter the name =”)

print (“ welcome” + name)

O/p - enter the name = WEST BENGAL

Welcome WEST BENGAL

Example- #python program for addition

X= input (“ Enter first Number :”)

y= input (“ Enter Second Number :”)

x= int (x)

y= int (y)

z= x + y

print(“sum is :” z)

Output- Enter first number 10

Enter second number=10

Sum is 30

Note- input () always given us string so whenever we need mathematical expression we have to covert it from string.

➤ Type () in python –

Type is a build – in function that tells you data type of a variable or a value.

Eg- **a = 10**
 Print (type(a))
 B=10.5
 Print (type(b))
 C= d + 3i
 print(type(c))

o/p- <class, 'int'>
 <class, 'float'>
 <class, 'complex'>

- In python we use 'none' to indicate that we don't know the value gat et c
 we did not assign any value to this variable .
- In python there is no char type , if you want you should create single hat
 String .

E/g- str = "gfg" (string is a sequence of charge)
 print(type(str))

J=[10,20,30]
print(type (j))
(array and dynamically add remove items. Items are stored in continuous location.)

T = (10, 20, 30)
Print (type (t))

(11 is also like a list, the difference is your can't Modify once you created A tuple. These are immutable)

S = {10, 20, 30}
Print (type(s))

(set is a collection of items where all items are distinct And set is like mathematical set)

D = { 10: "gfg", 20, "ide"}
Print (type(d))

(dictionaries are used to do mappings. This is a collection Of key: value pairs. Eg – items: prices, roll no: name.

➤ Type conversion in python -

Implicit

Eg- a=10
B= 1.5
c=a+b
print(s)
d=true
e= a+b
print(e)

o/p- 11.5
11

Explicit

Eg- s ="135"
FJ==1fl+oiantt((ss))
Print(i)
Print(f)

o/p – 145
135.0

Eg- & = "geek"
Print(list(s))
Print(tuple(s))
Print (set (&))

o/p - ['g', 'e', 'e', 'k', 's',]

Eg -3 a=20

Print (bin(a))

Print (hex(a))

Print (oct(a))

o/p- 0b10100

0*14

0024

➤ Comments in python –

We may wish to describe the code we develop we might. we should take notes of why select of script FM for instance formula procedure and sophisticated business login explained with command.

Single line command-

Multi- line comment - # or “”

Doc – string the string enclosed in triple quol-es comes immediately after the defined fn are called python Docstring.

➤ If else and elif in python –

There come situations in real life when we need to do some specific task and based on some specific conditions and , we decide what should we do next. Similarly , there comes a situation in programming where a specific task is to be performed if a specific condition is true.

In such cases , conditional statement can be used. And the conditional statement are –

- if
- if...else
- Nested if
- if-elif statement.

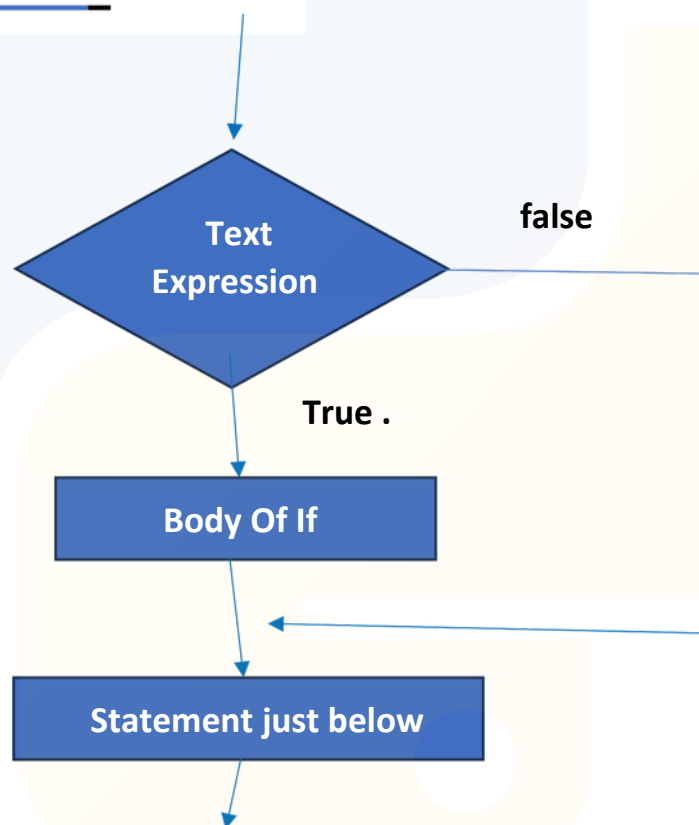
➤ If – statement - in this if the condition is true then if part executes else the part executes .

Syntax- if condition

statements to execute if

condition is true

➤ Flowchart -



Example – if $10 > 5$:

Print (“10 is greater then 5”)

Print (“program ended”)

o/p - 10 is greater then 5
program ended.

- **If else statement** - in condition if statement the additional block of code is merged as else statement which is performed when if condition is false .

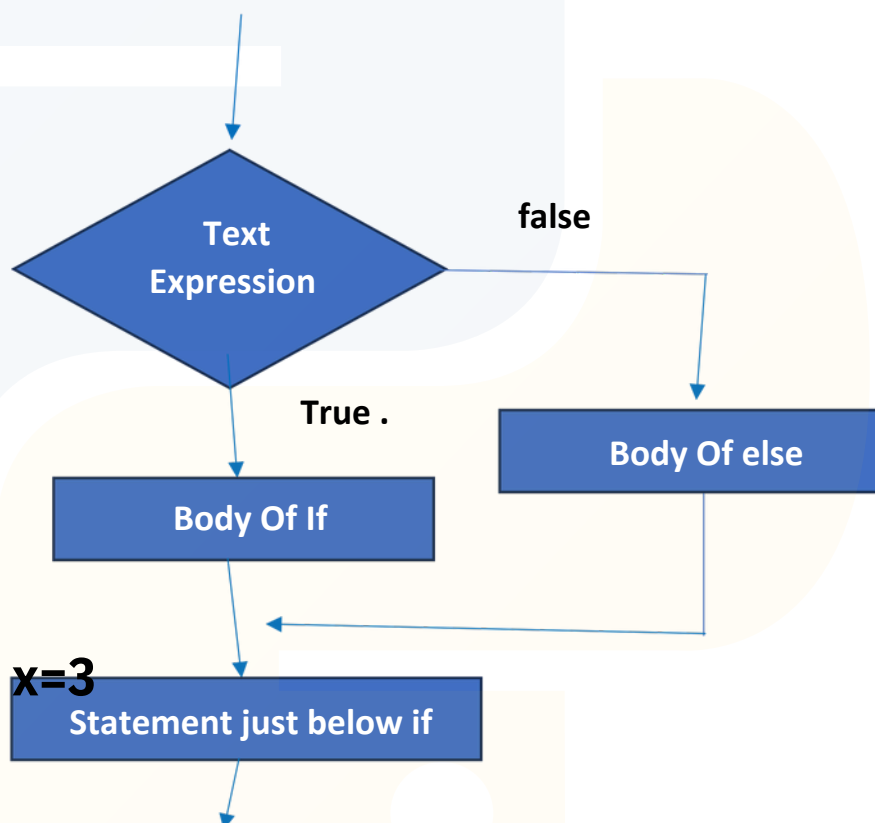
Syntax – if (condition)

**# executes this block if
condition is true**

else:

**# Executes this block if
condition is false**

. Flowchart -



Ex ~~x~~ a=m ~~p4l~~ ~~e~~ - ~~x=3~~

Print("yes")

Else:

Print("no")

Example – you can also chain if else statement with more than one condition .

Latter = "a"

If later == "B":

Else:

If later == "c"

print (" latter is c")

else :

```
If latter == "A":  
print (" latter is A ")  
Else  
print ("latter is "t A, B and C" )  
o/p – latter I A
```

- **Nested if statement** - if statement can also be checked inside other if statement. This conditional statement & called a nested if statement. This means that inner if condition will be checked only if outer if condition is true rue and and by this we can see multiple conditions to be satisfied.

Syntax - **if (condition 1) :**
 # executes when condition is true
 If (condition 2)
 # Executes when condition 2 is true
 # if block is end here
 # if block is end here

```
Example – num = 10  
If num >5:  
print (" bigger then 5")  
If num <=15:  
print ("between 5 and 15" )
```

o/p - bigger then 5
between 5 and 15

- **if – elif statement** - shortcut of if -else chain , while using if- else statement at the end else

block is added which is performed if none of the above if -else statement is true.

syntax - if (condition):

statement

elif(condition):

statement

.

else :

statement

Example – latter == “A”

If latter == “b”

Print (“latter is b”)

Elif latter==”c”

Print(“latter is c”)

Elife latter == “A”

else :

print(“latter isn’t A,B or C”)

o/p- latter is A

- **Arithmetic operators** – They are used to perform mathematical operations like additional, subtraction, multiplication , division.

.There are 7 arithmetic operators in python-

- | | | |
|-------------------|---------------|-------------------|
| 1. Addition | 2.Subtraction | 3. Multiplication |
| 4. Division | 5. Modules | 6. Exponentiation |
| 7. Floor division | | |

1. Addition operator- in python, + is the addition operator. It is used to add 2 values.

Example- val1 = 2

Val2 = 3

Res = Val 1+ Val 2

Print (res)

O/P – 5

2. Subtraction operator- Used to Subtract the Second Valu from the first Valu.

Example- Valu1 = 3

Valu2 = 2

res = val1-val2

print (res)

3. Modulus operator - % is the modulus operator used to find the remainder when first operand is divided by the second.

Example- val1 = 3

val2=2

res = val1 % val2

print (res)

O/P-1

4. Exponentiation operator - ** is used to raise the first operand to power of second.

Example- val1 = 2

val2 = 3

res = val1 ** val2

print(res)

O/P -8

5. Floor Division - // Is Used to Find the Floor of The Quotient When First Operand Is Divided By Second.

Example – Val 1 = 3

Val 2 = 2

Val 3 = Val 1 // Val 2

O/P – 1

➤ Python Logical Operator with Example –

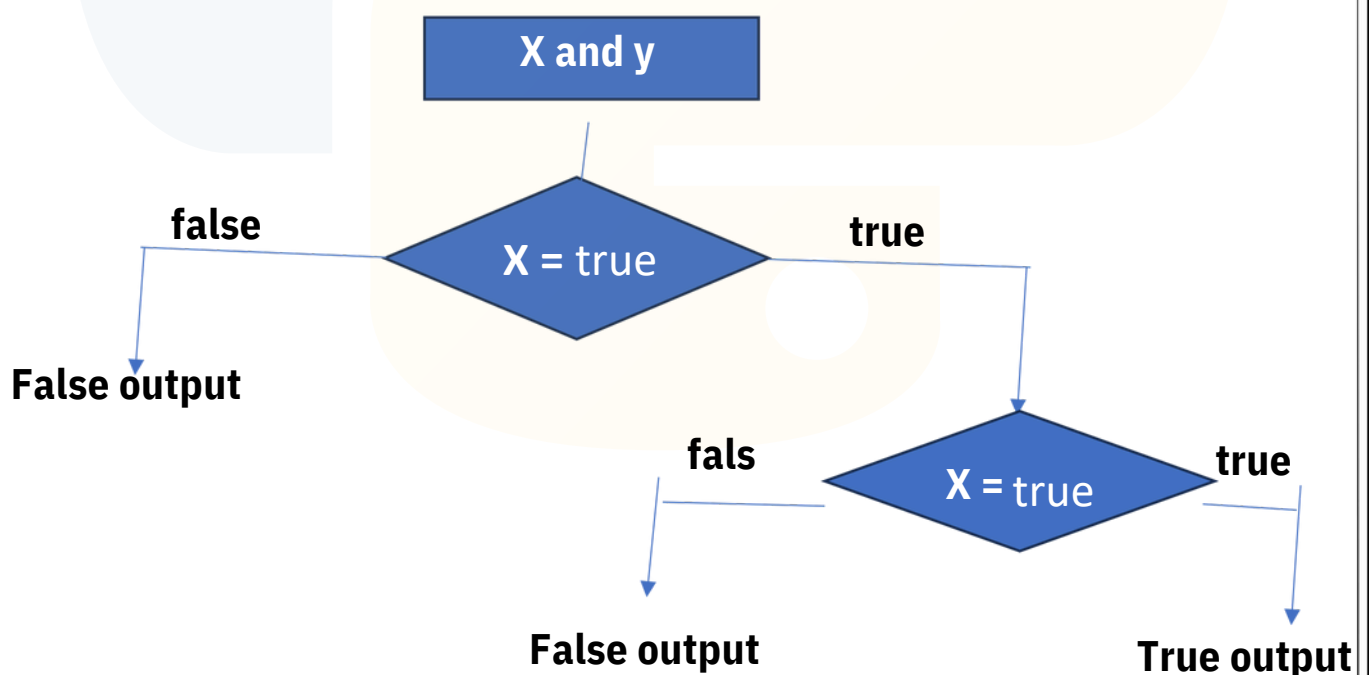
Operators Are Used to Perform Operation on Value and Variables. These Are the Special Symbols That Carry Out Arithmetic and Logical Computation. The Value the Operator Operates on Is Known as Operand.

-Logical And

- Logical Or

- Logical Not

-Logical And - True If Both the Operands Are True.



Example - A = 10

B = 10

C = 10

If a>0 And b>0:

print (“ The number are greater then o”)

If a>o and b>o and c>o:

print (“ The number are Greater then o”)

else:

print (“ Atleast one number is not greater then o”)

o/p- the number is not greater then o

atleast one number is not greater then o .

Example- a=10

b=12

c=0

If a and b and c :

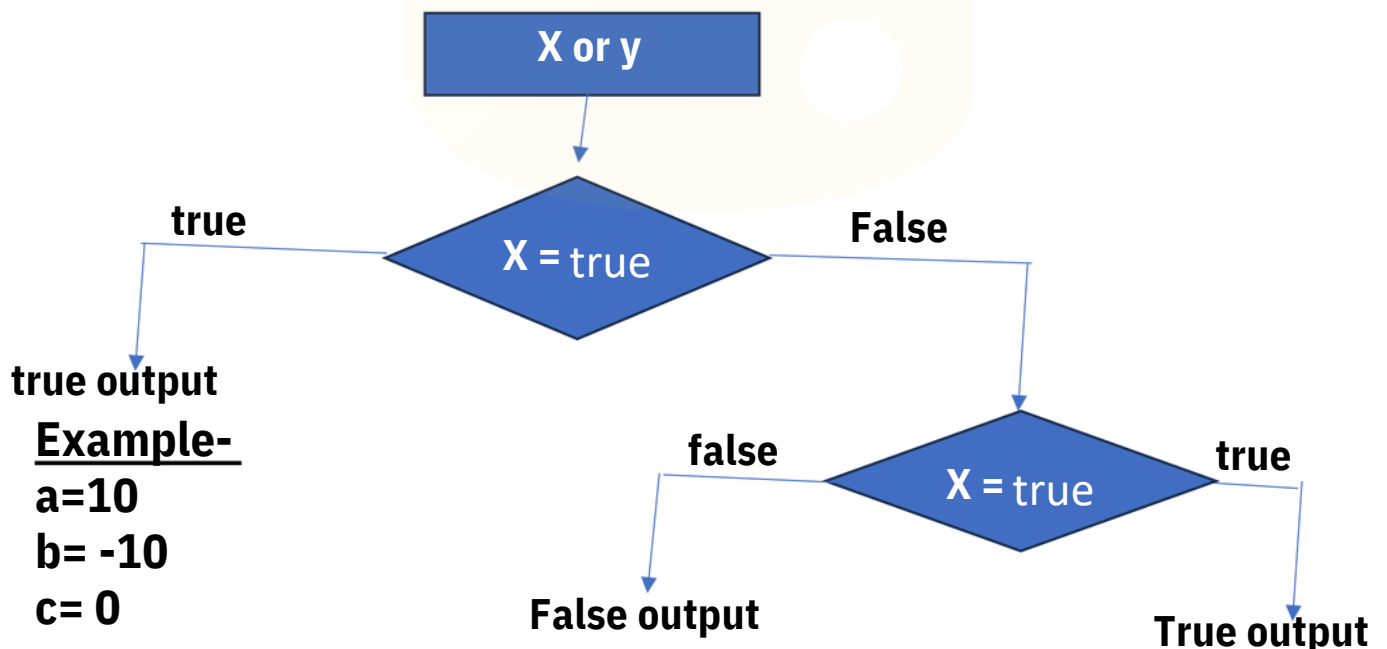
print (“All the number have Boolean value as type”)

else:

print(“Atleast one number has Boolean value as False”)

o/p – Atleast one number has Boolean value as False.

- Logical Or - if returns True if either of the operands is true.



if a>0 or b>0:

print ("Either of the number is greater than 0")

else:

print ("No number is greater than 0")

if b>0 or c>0:

print ("Either of the number is greater than 0")

else:

print ("No number is greater than 0")

O/P- Either of the number is greater 0

No number & greater than 0

Example - a = 10

b=12

if a or b or c:

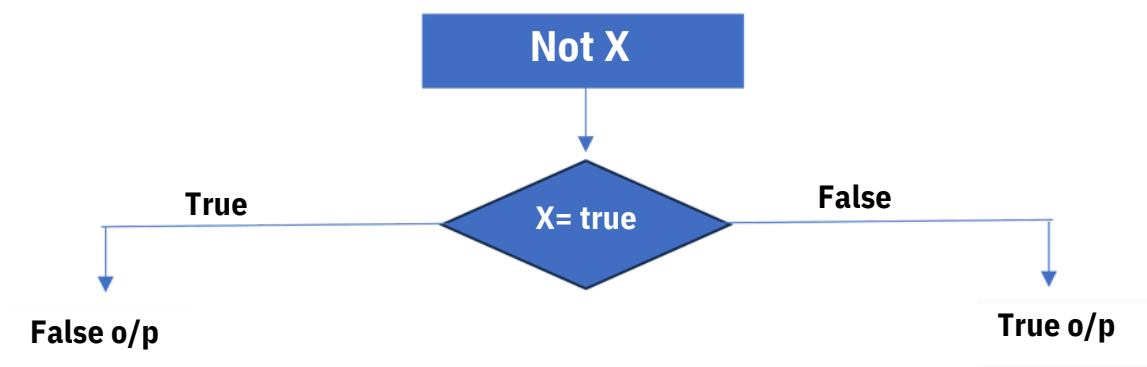
print("Atleast one number has boolean value true")

else:

print("All the numbers have boolean value as False")

O/P- Atleast one number has boolean value as True

- **Logical NOT Operator – working with a single Boolean value . if the boolean value is true it returns false and viceversa.**



Example- a= 10

```
if not a:  
    print ("Boolean value of a is True")  
if not (a%3== 0 or a%5=0):  
    print ("10 & not divisible by either 3 or 5")  
else:  
    print("10 & divisible by either 3 or 5")
```

o/p- 10 is divisible by either 3 or 5")

➤ **Onder of evaluation of logical operator-**

In the case of multiple operators, Python always evaluates the expression from left to right.

Example - def order(x):

```
print("Method called for value:", x)  
return True if x>0 else False  
a= order  
b = order  
c= order  
if a (-1) or b(5) or c(10):  
    print(" Atleast one of the number is positive")
```

O/p- Method called for value = -1

Method called for value = 5

Atleast one of the number is positive.

➤ Python Membership and Identity Operators-

Python offers two membership operators to check or validate the membership of a value. It tests for membership in a sequence such as strings, lists or tuples.

in operator The 'in' operator is used to check if a character / substring/element exists in a specified element in a sequence otherwise False For eg.-

'G' in 'GeeksforGeeks! # Checking 'G' in string

True

'g' in 'GeeksforGeeks' # Checking 'g' in string since python is case-sensitive, return False ,

False

Example- list1 =(1,2,3,4,5)

list2=(6,7,8,9)

for item in list1:

if item in list2:

print("overlapping")

else:

print ("not overlapping")

o/p- Not 100

Not

Not

Not

Not

- **'not in' operator** - Evaluates to true if it does not find a variable in the specified sequence and false otherwise.

Example - x=24

y=20

list = [10, 20, 30, 40, 50]

if (x not in list):

 print ("x is NOT present in given list")

else:

 print ("x is present in given list")

if (y in list):

 print("y is present in given list")

else :

 print("y is not present in given list")

o/p- x is not present in given list.

y is present in given list

- **Identity operators**- They are used to compare the object if both the objects are actually of the same data type and share the same memory location.

There are different identity operators such as

- **'is' operator** - Evaluates to true if the variables on either side of the operator point to the same object and false otherwise.

Example-

x=5

y =5

print(x is y)

id (x)

id(y)

o/p- 140704586672032
140704586672032

There, in the given Example, both the variable x and y have values assigned to it and both share the same memory location, which I why return True.

'is not' operator- Evaluates to false if the variables on either side of the operator point to a different object and true otherwise.

Example- x=5

```
If(type(x) is not int):  
    print ("true")  
else:  
    print("false")  
#print true  
X=5.6  
If (type(x)is not int ):  
    Print("true")  
else:  
    Print ("false")
```

o/p- false
True

➤ Python Bitwise Operators –

Operator are used to perform operation on values and variables. These are special symbols that carry out arithmetic and logical computations. The value the operator operates on is known as operand.

① Bitwise operators -

a)- Bitwise AND (&) - Return 1 if both the bits are 1 else 0.

Example -

a = 10 = 1010 (Binary)

b = 4 = 0100 (Binary)

a & b = 1010

&

0100

= 0000

= 0 (Decimal)

b)- Bitwise OR (|) - Returns 1 if either of the bits is 1 else 0.

Example -

a = 10 = 1010 (Binary)

b = 4 = 0100 (Binary)

a | b = 1010

0100

|

= 1110

= 14 (Decimal)

c) Bitwise NOT operator (~) - Returns one's complement of the no.

Example -

a = 10 = 1010 (Binary)

~a = ~1010

=(1010+1)

=(1011)

= -11 (Decimal)

d) Bitwise XOR (^) - Return 1 if one of the bits is 1 and the other is 0 else return 0.

Example -

a = 10 = 1010 (Binary)

b = 4 = 0100 (Binary)

a ^ b = 1110

^

0100

1110

=14 (Decimal)

Example -

a = 10

b = 4

#Print bitwise AND operation

print ("a & b = ", a&b)

Print bitwise OR operation

print ("a/b=", a/b)

#Print bitwise NOT operation

Print(" ~a=", ~a)

#Print bitwise XOR operation

print ("a^b =", a^b)

0/p- a & b = 0 ,

a / b = 14

~a=-11 a^b=14

Shift Operators – These operators are used to shift the bit of a number left or right thereby multiplying or dividing the no. by two resp .they can be used when we have to multiply or divide a number by two.

a) Bitwise Right Rep Shift (>>) - Shifts the bits of the no. to the right and fills 0 on void left (fills 1 in the case of a negative number) as a result. Similar effect as of dividing the no. with some power of two.

Example - a=10=0000 1010 (Binary)

a>>1=0000 0101 =5

Example – a=-10=1111 0110(Binary)

a>>1=1111 1011=-5

b) Bitwise Left Shift - (<<)

Shifts the bits of the number to the left and fills 0 on void right as result . Similar effect as of multiplying the no. with some power of two.

Example - a=5= 0000 0101 (Binary)

a<<1=0000 1010 =10

a<< 2=0001 0100=20

Example – b=-10=1111 0110(Binary)

b<<1 = 1110 1100 = -20

b<<2=1101 1000 =-40

3. Bitwise Operator Overloading - Operator Overloading
means giving extended meaning beyond their predefined operational meaning. For example operator + is used to add two integers as well as join two string and merge two lists.it is achievable because the '+' operator is overloaded by int class and str class. you might have noticed that the some built – in operator or funcation shows different behavior for object of different classes, this is called operator overloading.

Example - class Geek():

def-init-(self, value):

self. value = value

def-and-(self, obj):

print("And operator overloaded")

```

if isinstance (obj, Geek):
    return self.value & obj.value
else:
    raise value Error ("Must be a object of class Geek")
    def-or-(self, obj):
    print ("Or operator overloaded")
    if & isinstance (obj, Geek):
        return self. value / obj. value
    else:
        raise value error ("Must be a object of class Geek")
def -xor-(self, obj):
    print ("XOR operator overloaded")
    if isinstance (obj, Geek):
        return self. value ^ obj. value
else:
    raise value error ("Must be a object of class Geek")
    def-ishift-(self, obj):
    print("ishift operator overloaded")
    if isinstance (obj, Geek):
        return self. value <<< obj. value
else: raise value error("Must be a object of class Geek")"

def-invert-(self):
    print ("Invert operator overloaded")
    return~ self. Value
# Driver's code
If-name == "_main_":

    a = Geek (10)
    b= Geek (12)
    print (a & b)

```

```
print (a/b)
print (a ^ b)
print (a<<b)
print (a>>b)
print (~a)
```

0/b- And operator overloaded.

8

Or operator overloaded

14

Xor operator overloaded

8

Ishift operator overloaded

40960

rshift operator overloaded

8

Invert operator overloaded

-11

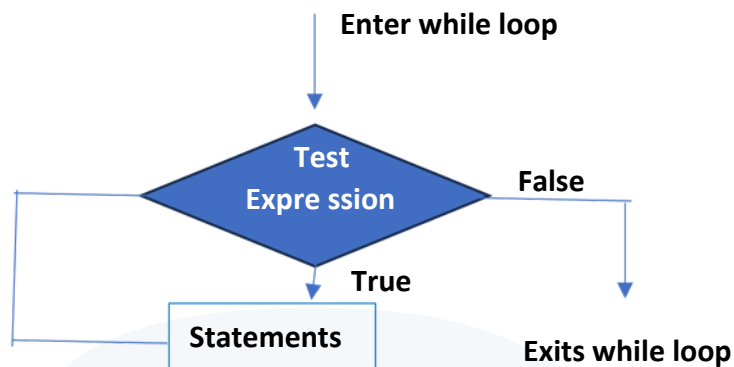
*** Loops in Python-**

- **While Loop** - It is used to execute a block of statements repeatedly until a given condition is satisfied. And when the condition becomes false, the line immediately after the loop in the program is executed.

Syntax- while expression:

statement(s)

Flowchart



While loop falls under the category of indefinite iteration. Indefinite iteration means that the number of times the loop is executed isn't specified explicitly in advance.

Statements represent all the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements. When a while or a do-while loop is executed, the expression is first evaluated in a boolean context and if it is true, the loop body is executed. Then the expression is checked again; if it is still true, the body is executed again and this continues until the expression becomes false.

Example:- 1 `count = 0`
 `While (count<3):`
 `Count = count + 1`
 `print ("hello geek")`

o/p- `Hallo Geek`
 `Hello Geek`
 `Hello Geek`

Example - 2

a=[1,2,3,4]

While a:

Print (a. pop())

o/p- 4
 3
 2
 1

In this , we have run a while loop over a list that will run until there is an element present in the list,

Example-3 single statement while block

Just like if block , if block , if the while block consists of single line . if there are multiple statement in the block that make up the loop body , they can be separated by semicolons (;) .

Count = 0

while (count<5): count + = 1; print (“Hello Geek”)

O/p - **Hello Geek**
 Hello Geek
 Hello Geek
 Hello Geek
 Hello Geek

-range () in python -

The python range () fn returns a sequence of number , in a given range . the most common use of it is to iterate sequence on a sequence of number using python loops.

Syntax – range (start, stop, step)

Parameter - start:[optional]

Stop:

Stop:

Return:

Example – for I in range (5):

print (I, end = “”)

print()

o/p - 0,1,2,3,4

What is the use of the range() ion python ?

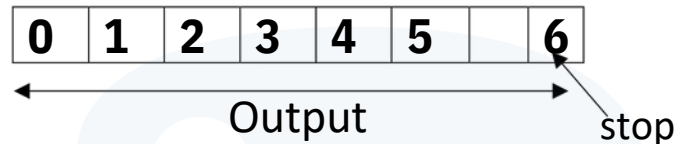
In simple terms , range () allows the user to generate a series of no . within a given range . Depending on how many arguments the user is passing to the fn, the user can decide where that series of number will being & end as well as how big the difference will be between one no. and the next. Python range() fn takes can be initialized in 3 ways –

- range (stop) takes one argument.
- range (start, stop) takes two argument.
- range (start, stop, step) takes three argument.

a. range (stop) -

When the User call range one argument, the user will get a series of number that starts at 0 and includes every whole number up to, but not including , the number that the user has provided as the stop

Example - python range (6)



Example - for I in range (6)

print (I , end ="")

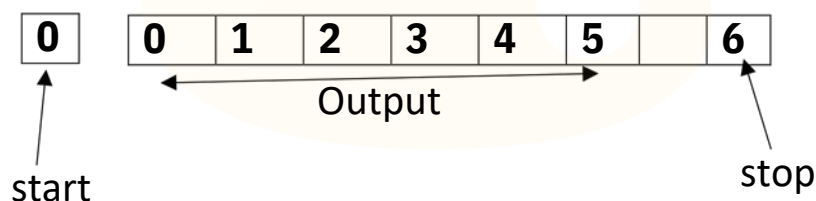
print()

o/p- 0 1 2 3 4 5

b. range (start, stop)-

when the user call range() with two argument, stop but also where it starts , so the user don't have to start at all the time . user can use range () to generate a series of number from X to Y.

python range (0,6)



Example - Demonstration of python range (start , stop)

printing a natural

number from 5 to 20

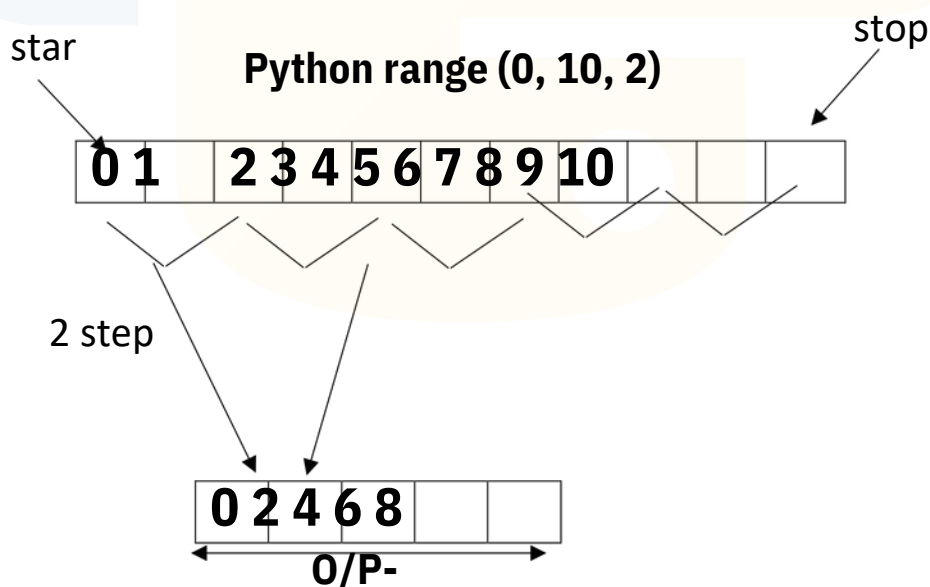
for I in rang (5, 20):

print(I , and = "")

o/p- 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

c. range (start, stop, step)

when the user called range () with there aruments, the user can choose not only where the series of number will start and stop , but also how big the different will be between the step , then range() will automatically behave as if the step is 1. in this example , we are printing even no . between 0 and 10 so we choose out string point from 0 (start =0) and stop the serics al 10 (stop = 10). For printing on even number the difference between one number and the next number must be 2 (step =2) after providing a step we get the following output (0, 2, 4, 6, 8)



Example – Demonstration of python range (start, stop ,step)

```
for I in range (0,10,2):
```

```
print (I, end= “”)
```

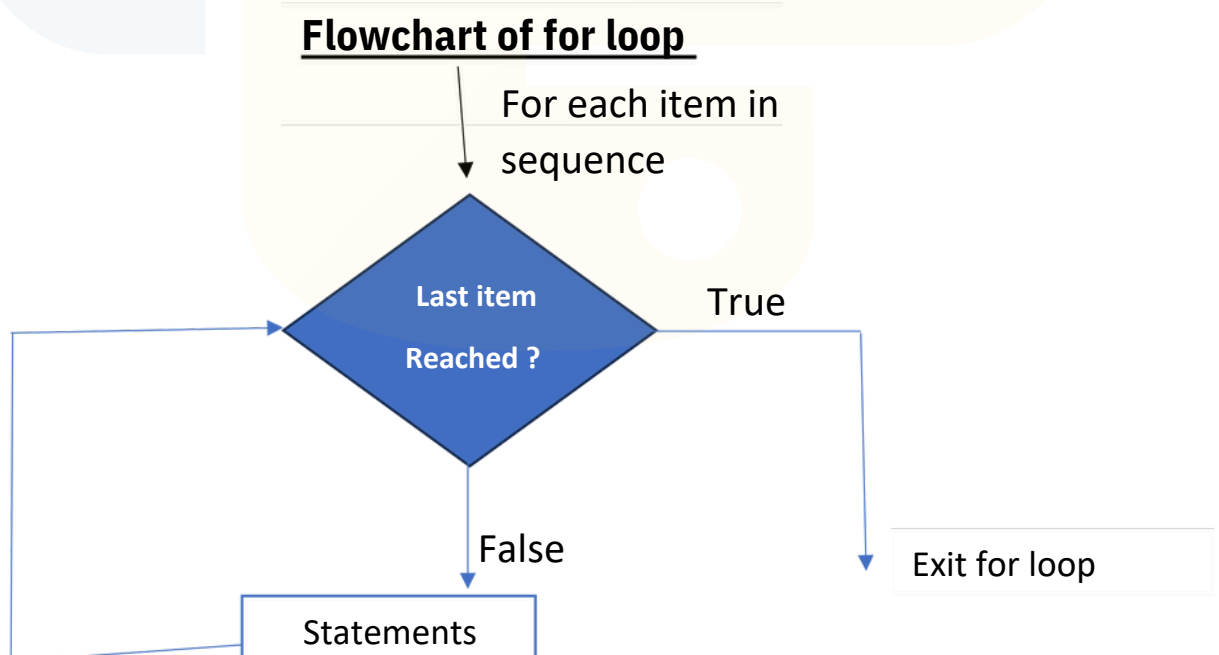
```
print()
```

o/p- 0 , 2, 4 , 6 , 8

- For Loop In Python - it is used for sequential traversal i.e., it is used for iterating over an iterable like string, tuple, list, set or Dictionary. In python there is no c style for loop, i.e., for (i=0; i<n; i++). There is a loop which is similar to each loop in other languages.

Note- in python, for loop only implements the collection – based iteration.

Syntax - **for var in iterable :**
 # statement



Here the iterable is a collection of objects like lists, tuples. The indented statements inside the for loop are executed once for each item in an iterable. The variable `i` takes the value of the next item of the iterable each time through the loop.

Example-

```
# Python program to illustrate
#iterating over a list
1= ["geeks", "for", "geeks"]
for i in 1:
print(i)
o/p- geeks
      for
      geeks
```

Example - Loop in Dictionary

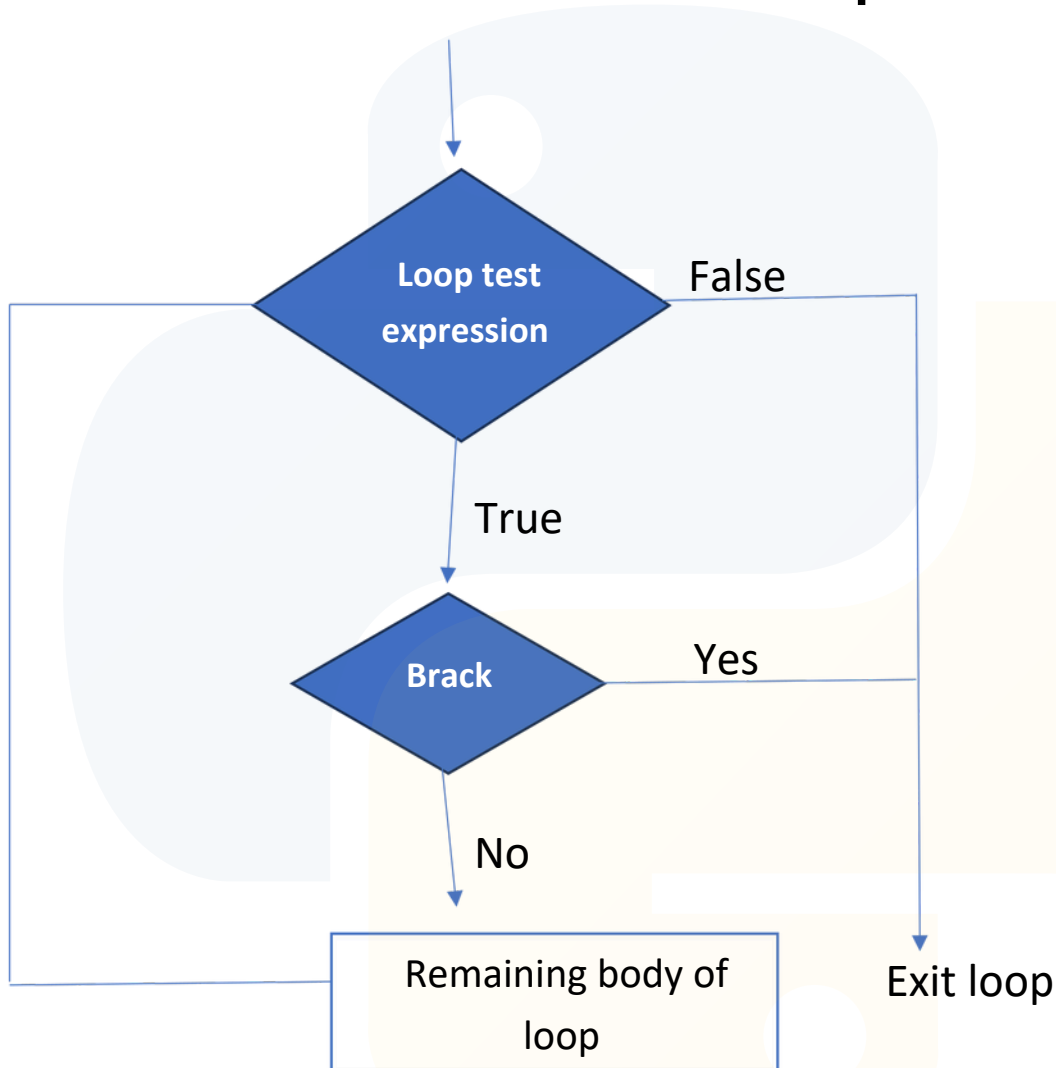
```
#iterating over dictionary
print (Dictionary Iteration")
```

```
d= dict()
d['x y z'] = 123
d[a b c] = 345
for i in d:
print("%s %d" % (i, d(i)))
```

O/P- Dictionary Iteration

xyz	123
abc	345

break in python- It is used to bring the control out of the loop when some external condition is triggered. break statement is put inside the loop body (generally after if condition). It terminates the current loop , i.e., the loop in which it appears, and resumes execution at the next break statement is inside a nested loop , the break will terminate the innermost loop .



Example - for I in range (10)
print (i)
If I ==2:
brack

o/p- 0
1
2

Example-2 s= 'geeksforgeeks'
#using for loop

```

        for letter in s;
        print (letter)
    #break the loop as soon it sees 'e'
    # or 's'
        if letter == 'e' or letter == 's'
        break
        print ("Out of for loop")
        print()
        i:0

```

```

#Using while loop
    while True:
        print (s[i])
    #break the loop as soon it sees 'e'
    # or 's'
        if s[i] == 'e' or s[i]=='s':
            break
        i+= 1
    print ("Out of while loop")

```

o/p- g

e

Out of for loop

g

e

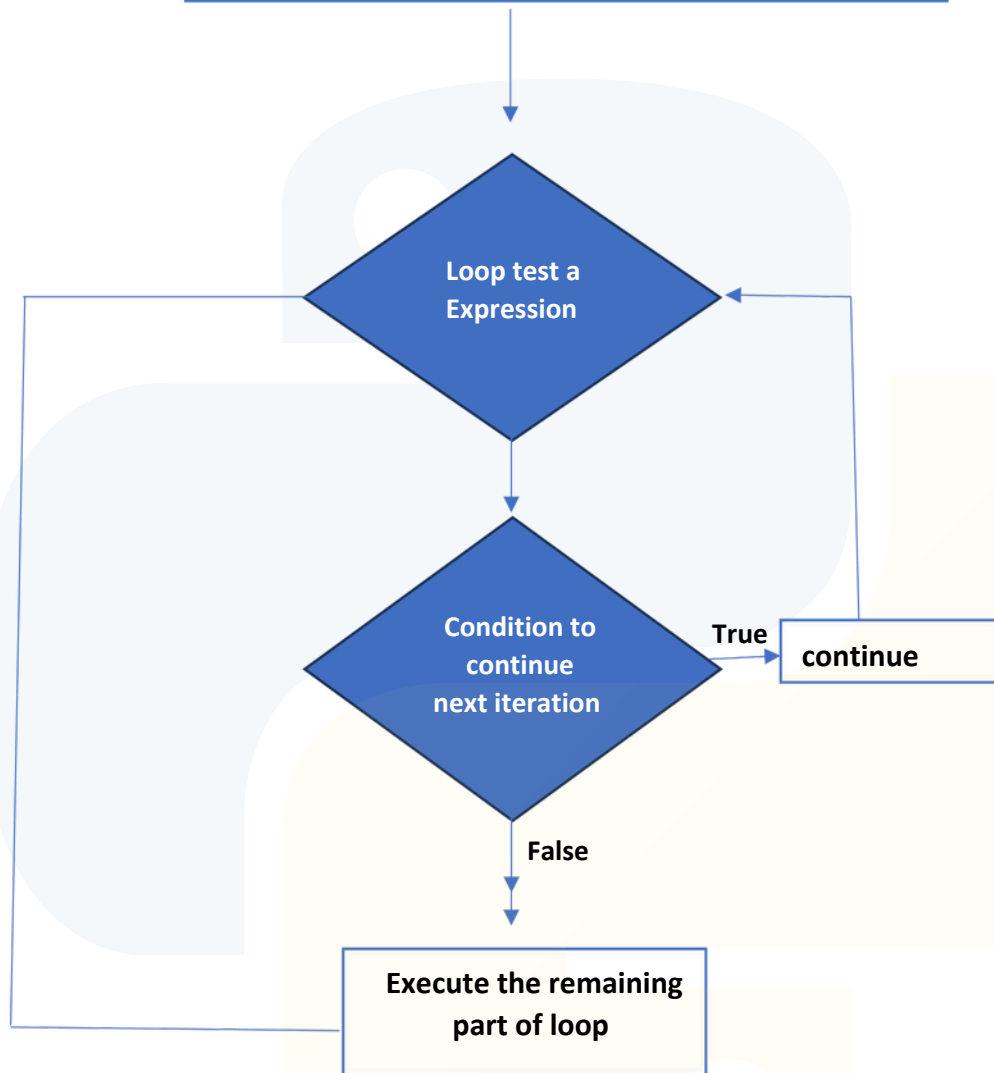
Out of while loop

* [Continue Statement in Python](#) - It is a loop control

statement that forces of execute the next iteration of the loop while skipping the rest of the code inside the loop for current iteration only, i.e, when the continue statement is executed in the loop , the code inside the loop following the continue statement will be skipped for the current iteration and the next iteration of loop will begin.

Syntax- while true:
If x == 10:
Continue
print(x)

Flowchart of continue statement -



Example - for var in “Geeksforgeeks”;
If var == “e”:
Continue
print(var)

o/p- g

k

s

f

o

r

g

k

Explanation- there we are skipping the print of character using if-condition checking and continue statemen.

Example-2

Consider a situation when you need to write a program which prints the number from 1 to 10, but not 6.

```
# Joop from 1 to 10
for i in range (1,11)
    # if i is equal to 6,
    # continue to next iteration
    # without printing
    if i==6:
        continue
    else:
        # otherwise print the value
        # of i
        print (i, end = "")
```

0/b - 1 2 3 4 5 7 8 9 10

Note - The Continue Statement Can Be Used With Any Other Loop Also Like "While Loop". Similarly As It Is Used With 'For Loop' Above.

*** Nested Loop -** with for and while loops we can create nested loop . nested loop means loops inside a loop. For example , while loop inside the for loop , for loop inside the loop , etc.

Syntax - Outer loop Expression :

Inner- loop Expression:

Statement inside outer loop

Example - x= [1,2]

Y = [4,5]

For I in x :

For j in y :

print (I , j)

o/p - 14

15

24

25

Example –

Multiplication Table

Running outer loop from 2 to 3

For I in rang (2,4):

printing inside the outer loop

running inner loop from 1 to 10

For j in range the inner loop

print (I , “x”, j “=” , I * j)

printing inside the other loop

print()

o/p - 2*1=22*10=20

3*1 =3..... 3*10= 30

Example -

```
# Initialize list and list 2.  
# with some strings  
list1: ['I am', 'You are']  
list 2: ['healthy', 'fine', 'geek']  
# Store length of list 2 in list 2-size  
list 2. size len (list 2)  
# Running outer for loop to  
# iterate through a list 1  
for item in list 1:  
    # Printing outside inner loop  
    print("start outer for loop")  
    # initialize counter i with 0  
    i = 0  
    # Running inner while loop to  
    # iterate through a list 2  
    while (i < list 2 size):  
        # Printing inside inner loop  
        print (item, list 2[i])  
        # Incrementing the value of i  
        i=i+1  
    # printing outside inner loop  
    print ("end for loop")
```

O/p- start outer for loop

```
I am healthy  
I am fine  
I am geek  
end for loop  
start outer for loop  
You are healthy  
You are fine  
  
You are geek  
End for loop
```

In this example , we are initializing two lists with some string. Store the size of list 2 in list 2- size using len() functioning and using it in the while loop as a counter. After that run on outer for loop to iterate over list 2 using list indexing that we are printing each value of list 2 for every value of list 1 .

- Function in python –

Python function is a block of statement that return the specific task.

The idea is put some commonly or repeatedly done tasks together and make a function so that instead of writing the some code again and again for different input , we can do the function calls to reuse . code contained in it over and over again.

Syntax - python function

Keyword	Function name	Parameters
<u>def</u>	<u>function name</u>	<u>(parameters):</u>

statement] → Body of Statement

return expression

↑

function return

Example- def fun():
 print("Welcome to GFG")
 #Driver code to call a function
fun()

O/p- Welcome to GFG.

- Defining and calling a function with parameters-

Syntax- def function-name(parameter: data-type)
 return-type:
 """Doctring"""
 # body of the function
 return expression

Example - # some more functions

```
def is-prime (n):  
    if n in [2,3]: return True  
    if ( n !=1) or (n% 2 ==0) :  
        return False  
    r = 3  
    while u*u <= n  
    if n % u ==0 :  
        return False  
    u+=2  
    return true  
print (is - prime (78), is prime (79))
```

O/p - False True

➤ Arguments of a Python Function -

Arguments are the values. Pass inside the paranthesis of the function. A function can be have any be number of arguments separated by a comma

Example - # A simple Python Function to check

```
# whether x is even or odd
def even odd(x):
    print("even")
    else:
    print ("odd")
#Driver code to call the function
even Odd (2)
even Odd (3)
```

o/p- even
odd

1-Types of Arguments-

- 1. Default argument** - It is a parameter that assumes a default value if a value is not provided in the function call for that argument.

Example - def my Fun (x , y=50):

```
print("x:",x)
print("y;", y)
```

**#Driver code (we call my Fun () with
#only argument)**

my Fun (10)

**o/p- x=10
y=50**

1. Keyword argument –

The idea is to allow the caller to specify the argument name with values so that caller does not need to remember the order of parameters.

Example – def student (first name, last name):

print (first name, last name):

keyword arguments.

Student (first name = 'geeks', last name = 'for')

Student (last name = 'for', first name = 'geeks')

o/p – geeks for

geeks for

2. Variable- length arguments –

We can pass a variable number of arguments to a function using special symbol. There are two special symbol –

. * args (non-keyword arguments)

. ** kwargs (keyword arguments)

Example - def my fun (*argv):

For arg in argv:

print(ary)

My fun ('hello', 'welcome', 'to', geeks for geeks')

o/p – hello

welcome

to

geeks for geeks

- **Docstring** – The First String After the Function Is Called the Document String or Docstring in Short. This Used to Described the Functionality of The Function. The Used for Docstring tn Fn Is Optional but It Is Considered a Good Practise.

Syntax – Print (Function_ Name. _ Doc_)

Example - Defevenodd (x):

““Function To Check Whether X Is Even or Odd”””

If (X % 2 == 0):

print (“Even”)

else:

print (“Odd”)

#Driver Code to Call the Function

Print (Even Odd. _Docs_)

O/P – Function to Check If the Number Is Even or Odd.

- **Return Statement in Python Function** –

The Function Return Statement Is Used to Exit from A Function and Go Back to The Function Caller and Return the Specified Value or Data Item To The Caller.

Syntax - Return [Expression_ List]

The Return Statement Can Consist of a Variable. An

Expression, or a constant which is returned to the end of the function execution. If none of the above is present with the return statement a none object is return.

Example – `def square_value (num):`

`“““this function Return the square value
 Of the entered number”””`

`print (square_value (2))`

`print (square_value (4))`

o/p - 4

 16

-pass by reference or pass by value –

One important thing to note is, in python every variable name is a reference. When we pass a variable to a function, a new reference to the object is created.

Example - `def my fun (x):`

`x [0] = 20`

`# driver code (note the 1st is modified after)`

`# function call`

`1st = [10, 11, 12, 13, 14, 15]`

`My fun (1st)`

`Print (1st)`

o/p - [20, 10, 11, 12, 13, 14, 15]

- String in python – String Are array of bytes representing Unicode characters.

Example – “Geeks for Geeks” Or ‘Geeks for Geeks’

Python Does Not Have a Character Data Type, A Single Character Is Simply a String with A Length Of 1. Square Brace Can Be Used to Access Elements in String.

**Example- Print (“A Computer Science Portal for Geeks”)
O/P- A Computer Science Portal for Geeks.**

-Creating A String In Python-

String In Python Can Be Created Using Single, Double Or Even Triple Quotes.

**Example – String 1 = ‘Welcome to The Geeks World’
print (“String with The Use of Single Duotes:”)
print (String 1)
Creating A String
With Double Quotes
String1 = “I Am Geek”
print (“In String with The Use of Double Quote”)
print (String1)
Creating A Sting with Triple Quotes
String = “I’m A Geek”
print (“In String with The Use of Triple Quotes:”)

Print (String1)
Creating String with Triple
Quotes Allows Multiple Lines
String1= “Geeks
For
Life”
Print (“In Creating A Multiline Sting:”)**

```
print (strin1)
```

O/P - String with the use of single quotes:

Welcome to the geek world

String with the use of double quotes:

I'm a geek

String with the use of Triple quotes:

geeks

For

life

-Accessing character in python string- In python, individual

characters of a string can be

accessed by using the method of Indexing. Indexing allows negative address references to access characters from the back of the string, eg.1 refers to the last char, -2 refers to the second last char and so on.

While accessing an index out of the range will cause an Index Error. Only Integers are allowed to be passed as an index. Float or other types that will cause a Type Error.

Example- String1 = "geeks for geeks"

Print ("Initial string:")

Printing First character

Print ("In First character of string is:")

Print(string1[0])

printing Last Character

print (“In Last Character Off String Is:”)

Print (String 1 (-1))

O/P – Initial String:

Geeks For Geeks

First Character of String Is:

G

Last Character of String Is:

S

-Reversing A Python String –

Program To Reverse a String

Gfg = “Geeks for Geeks”

Print (Gfg[: :-1])

O/P – Skeegrofskeeg

-String Slicing –

To Access a Range of Character In The String, The Method Of Slicing Is Used. Slicing In a String Is Done by Using a Slicing Operator (Colon).

Example – String 1 = “Geeks for Geeks”

Print (“Initial String:”)

Printing (String 1)

Print (“\N Slicing Character For 3 – 12:”)

Print (String 1 [3:12])

Print

(“\N Slicing Character Between”+ “3rd And 2nd Last Character:”)

Print (string 1 [3: -2])

o/p – initial string:

geeks for geeks

slicing character from 3-12:

ks for geek

slicing character between 3rd and 2nd last for char:

ks for gee

-chr () function –

It return a string from a Unicode code integer.

Eg - print (char (97))

o/p – a

• Escape sequencing in python –

While printing string with single and double quote in it cause syntax error because string already contains single and double quotes and hence cannot be printed with the use of either of these. Hence, to print such a string either triple quotes are used to escape sequences are used to print such string.

-string concatenation using + operator –

This operator can be used to add multiple string together. However, the arguments must be a string. Here, the + operator combines the string that is stored in the var1 and var 2 and stores in another variable var 3 .

Note – STRING ARE IMMUTABLE, THEREFORE, WHEN EVER IT IS CONCATENATED, IT IS ASSIGNED A NEW VARIABLE.

-Index () Method - It Allows A User To Find The Index Of The First Occurrence Of An Existing Substring Inside A Given String.

Syntax – String – Obj. Index (Substring, Begp, Endp)

Parameters - . Substring – The String To Be Searched For.

. Begp – (Default : 0) This Fn Specifies The Position From Where Search Has To Be

Started

. Endp (Default: Length of String): These Fn Specifies The Position from Where Search Has to End.

Return – Return The 1st Position of Substring Found

Exception – Raise Value Error If Argument String Is Not Found Or Index Is Out of Range.

Rindex () Method – It Returns the Highest Index Of The Substring Inside String If The Substring Is Found. Otherwise, It Raises Value Error.

➤ **Case Changing of String** –

. Lower () – Convert All Upper Case Characters In A String Into Lower Case.

. Upper () – Convert All Lower Case Character In A String Into Upper Case.

. Title () – Convert String To Title Case.

➤ Built In Methods – Start With () And Ends With ()
Syntax – Starts With () – Str. Starts With (Search- String,
Start, End)

End With () – Str. Ends With (Search, String, Start,
End)

-Python String Find() Method - It Returns the Lowest
Index

Or First Occurrence of The Substring If It Is Found in A
Given String. If It Is Not Found, Then It Returns – 1.

Syntax – Str_ Obj. Find (Sub, Start, End)



Substring that need
to be searched

• List Introduction –

Python List Are Just Like Dynamically
Sized Arrays, Declared in Other Language (C ++, Java). In
A Simple Language, It Is Aa Collection of Things, Enclosed
With [] And Separated by Commas.

The List Is a Sequence Data Type Which Is
Used to Store the Collection of Data. Tuples And Strings
Are Other Types of Sequence Data Types.

Eg - Var = ["Geeks", "For", "Geeks"]

print (Var)

O/P – ['Geeks', 'For', 'Geeks']

-Lists Need not to be homogeneous always which Makes It the Most Powerful Tool in Python.

-A Single List May Be Contain Data Type Likes Int, Str, As Well As Objects.

-List Are Mutable, And Hence, They Can Be Altered Even After Their Creation.

*** Creating A List in Python -**

List Can Be Created by Just Placing the Sequence Inside the Square Braces []. Unlike Sets, A List Doesn't Need a Built – In Function for The Creation of Aa List.

Example - List = []

```
print ("Blank List:")
```

```
print (*List*)
```

```
List [ 10, 20, 30]
```

```
print ("In List of Numbers:")
```

```
print (List)
```

O/P - Blank List:

```
[ ]
```

List Of Number:

```
[ 10, 20, 30]
```

-Accessing Element from The List –

```
List = ["Geeks", "For", "Geeks"]
```

```
print ("Accessing a Element:")
```

```
print (List [0])
```

```
print (List [2])
```


-Negative Indexing

-Getting The Size Of The List –

```
List 1 = [ ]
```

```
print (len (list 1 ))
```

```
List 2 = [ 1, 2, 3]
```

```
print (len (list 2))
```

o/p – 0

3

-Adding Elements To A List –

Using append () method –

-only 1 element can be added at a time, for the addition of multiple elements with the append () method, loops are used.

-tuple can be added to the list with the use of append method because tuple are immutable.

Example - list = []

```
print (“initial blank list:”)
```

```
print (list)
```

```
List. Append (1)
```

```
List. Append (2)
```

```
List. Append (4)
```

```
Print (“\n list after addition of three elements:”)
```

```
Print (list)
```

```
For I in range (1, 4):
```

```
List. Append (i)
```

```
print (“\n list after additional of elements from 1-3:”)
```

```
print (list)
```

List. Append ((5,6))

```
print (“\n list after additional of a tuple:”)
```

```
print (list)
```

List 2 = ['for', 'geeks',]

List. Append (list 2)

```
print (“\n list after addition of a list:”)
```

```
print (list)
```

o/p – initial blank list:

```
[ ]
```

List after addition of three element:

```
[1, 2, 3]
```

List after addition of element from 1- 3:

```
[1, 2, 4, 1, 2, 3]
```

List after addition of a tuple:

```
[1, 2, 4, 1, 2, 3, (5, 6)]
```

List after addition of a list:

```
[1, 2, 4, 1, 2, 3, (5, 6), ['for', 'geeks']]
```

-using insert () method –

It insert a given element at a given index in a list using python.

Syntax – list_ name . insert (index, element)

Index- at which the element has to be inserted.

Element – to be inserted in the list.

Returns – does not return any value.

Example –

```
Lis = ['geeks', 'for', 'geeks',]
```

```
Lis. Insert (1, "for")
```

```
print (lis)
```

o/p - ['geeks', 'for', 'geeks']

- [count \(\) method](#) - returns the count of how many times a given object occurs in a list.

Syntax – list_ name. count (object)

Parameters –

. object – is the item whose count is to be returned.

Exception –

. type error – raises type error if more than 1 parameters is passed in count () method.

Example - list 2 = ['a', 'a', 'a', 'b', 'b', 'a', 'c', 'b']

```
print (list 2. Count ('b'))
```

o/p – 3

- [del keyword](#) – del is a key word and remove (), pop () are in built method.

- remove () method delete value or object from the list using value.

- the del and pop () delete values or objects from list using and objects.

- the del keyword deletes any variable, or list of value from a list.

Syntax –

del list _ name[index] # to delete single value

del list _name # to delete whole list

-The remove () method removes the first matching value from list.

Syntax – list_ name. remove (value)

-The pop () method like the del keyword deletes value at a particular index. but the pop () method returns deleted value from the list.

Syntax – list_ name .pop(index)

Example – numbers = [1, 2, 3, 2, 3, 4, 5]

use remove() method

numbers . pop (3)

print (numbers)

use remove() method

numbers . pop (-1)

print (numbers)

numbers .pop(0)

print (numbers)

O/P – [1, 2, 3, 3, 4, 5]

[1, 2, 3, 3, 4]

[2, 3, 3, 4]

-max() method - used to compute the maximum of the values passed in its argument and the lexicographically largest value if string are.

Passed as arguments.

Example- print (“Maximum of 4,12,43.3,19 and 100 is :”, end= “”)

Print (max(4,12,43.3,19 100))

O/P _ Maximum of 4,12,43.3,19,100 is = 100

Mini () method- is used to compute the minimum of the values passed in its arguments.

Sort() function- can be used to sort a list in ascending, descending, or user- defined order.

Syntax- list_ name , sort (reverse= True/False, key= my func)

reverse() method- is a inbuilt method in python that reverses object of the list in place i.e., it doesn't use any extra space it just modifies the original list.

Syntax- list_ name, reverse ()

Tuples in python-

Python Tuple is a collection of objects separated by commas. In some ways , a tuple is similar to a list in terms of indexing, nested objects, and repetition but a tuple is immutable, unlike lists which are mutable.

To create a Tuple we will use () operators.

Var=(“Geeks”, “for”, “Geeks”)

Print(var)

-Accessing values in Tuples-

Method-1 using positive index

Method-2 using Negative index

Sets in python –

It is an unordered collection data type that is iterable, mutable and has no duplicate elements.

Sets are represented by {}

Since sets are unordered , we can't access items using indexes like we do in lists.

**Example- var = {"geeks," "for", "geeks"}
print(type (var))**

O/P- <class set>

-Frozen set -are immutable objects that only support methods and operators that produce a result without affecting the frozen set or sets to which they are applied. It can be done with frozen set() method.

-Methods for sets-

- Adding elements to sets – set add () function
- Union operation on sets - union () function
- Intersection operation - intersection () or &
operator - difference () or –
- Difference
operator
- clearing sets - clear () method

• Dictionary In Python –

It Is Collection Of Keys Values, Used To Store Data Value Like Aa Map, Which, Unlike Other Data Types Which Hold Only A Single Value As An Element.

-Dictionary Holds Key : Value pair.

**Example – Dict : {1 : 'Geeks', 2 : 'For', 3 : 'Geeks'}
print (Dict)**

-Grating A Dictionary –

A Dictionary Can Be Created By Placing A Sequence Of Elements Curly { } Brackets, Separated By 'Comma'. Dictionary Holds Pair Of Values, One Being The Key And The Other Corresponding Pair Element Being Its Value. Values In A Dictionary Can Be Of Any Data Types And Can Be Duplicated, Where As Keys Can't Be Repeated And Must Be Immutable.

Note - Dictionary Keys Are Case Sensitive, The Same Name But Different Case Of Key Will Be Treated Distinctly.

**Example – Dict = { 1: 'Geeks', 2: 'For', 3: 'Geeks'}
Print (“\N Dictionary With The Use Of
Inter Ger Keys:”)**

Print (Dict)

**Dict = {'Name', 'Geeks', 1:[1,2,3,4]}
print (“\N Dictionary With The Use Of Mixed Keys:”)
print (Dict)**

O/P – Dictionary With The Use Of Integer Keys :

{ 1: 'Geeks', 2: 'For', 3: 'Geeks' }

Dictionary With The Use Of Mixed Keys :

{ 'Name' : 'Geeks', 1: [1, 2, 3, 4] }

-Dictionary Can Also Be Created By The Built – In Function Dict (). An Empty Dictionary Can Be Created By Just Placing To Curly Braces {}.

-Nested Dictionary –

Dict = { 1: 'Geeks', 2: 'For', 3: {'A': 'Welcome',
'B': 'To', 'C': 'Geeks' } }

Print (Dict)

O/P – { 1: 'Geeks', 2: 'For' 3: {'A' : 'Welcome', 'B' : 'To', 'C' : 'Geeks'}}

-Adding Elements To A Dictionary –

One Value Can Be Added To A Dictionary By Defining Value Along With The Key . Eg – Dict [Key] = 'Value'.

Updating An Existing Value In A Dictionary Can Be Done By Using The Built- In Update () Method.

Nested Key Values Can Be Also Added To An Existing Dictionary.

-Accessing Element To A Dictionary – In Order To Access The Item Of A Dictionary Refer To Its Key Name. Key Can Be Used Inside Square Braces.

-there is also a method called `get ( )` method that will be help in accessing the element from a dictionary . this method accepts key as argument and return the value.

➤ Dictionary method –

- 1> `Clear ( )` – remove all the elements from the dictionary.
- 2> `Copy ( )` – return a copy of the dictionary.
- 3> `Get ( )` – return the value of specified key.
- 4> `Items ( )` – return a list containing a tuple for each key value pair.
- 5> `Keys ( )` – return a list containing dictionary keys
- 6> `Pop ( )` – remove the element with specified key
- 7> `Pop time ( )` – remove the last inserted key- value pair.
- 8> `Update ( )` – update dictionary with specified key-value pairs.
- 9> `Values ( )` – return a list of all the values of dictionary.

• Slicing (list , tuple and string –

Slicing a python is a feature that enables accessing parts of the sequence. In slicing a string, we create a substring, which is essentially a string that exists with in another string. We use slicing when we require a part of the string and not the complete string.

Syntax – string [start: end: step]

- Start – we provide the starting index.
- End – we provide the end index .
- Step – it is an optional argument that determine

The interment between each index for slicing.

-Slicing in list –

Lst = [50, 70, 30, 20, 90, 10, 50]

print (lst [1: 5])

o/p – [70, 30, 20, 90]

-slicing in tuple –

Tup = (22, 3, 45, 4, 2.4, 2, 56, 890, 1)

print (tup [1 : 4])

o/p – 3, 45, 4)

-slicing in tuple –

Tup = (22, 3, 45, 4, 2.4, 2, 56, 890, 1)

print (tup[1:4])

• Comprehension in python –

A python list comprehension consists of brackets containing the expression, which is executed for each element along with the for loop to iterated over is element in the python list.

-python list comprehension provides a much more short syntax for creating a new list based on the values of an exciting list.

syntax -

**new list = [expression (element) for element
in old list if condition]**

-list comprehension vs For loop-

There are various ways to iterate through a list. However, the most common approach is to use the for loop.

```
List = []  
for character in 'Geeks 4 Geeks!':  
    list.append(character)  
print(list)
```

O/P- ['G', 'e', 'c', 'k', 's', '4', 'G', 'e', 'e', 'k', 's', '!']

-python Dictionary comprehension-

Here we have two lists named keys and values and we iterating over them with the help of zip () function.

Keys = ['a', 'b', 'c', 'd', 'e']

Values =[1,2,3,4,5]

```
myDict = {k : v for (k , v) in zip (keys , value) }  
print (myDict)
```

O/P- { 'a':1, 'b' : 2, 'c':3, 'd':4, 'e':5 }

❖ Python oops concepts and class-

Oops is a programming paradigm that uses object and classes in programming. It aims to implement real-world entities like, inheritance, polymorphism, encapsulations etc. in the programming. The main concept of oops is to bind the data and the functions that work on that together as a single unit so that no other part of the code can access this data.

Main concepts of oops-

- Class**
- Objects**
- Polymorphism**
- Encapsulation**
- Inheritance**
- Data Abstraction**

-Class- A class is a collection of objects. A class contains the blueprints or the prototype form which the object are being created. It is a logical entity that contains some attributes and methods.

-Class- A class is a collection object . A class contains the blueprints or the prototype from which the objects are being created. It is a logical entity that contains some attributes and methods.

- Classes are created by keyword class.**
- Attributes are the variables that belongs to a class.**
- Attributes are always public and can be accessed using the dot(.) operator.**
- Eg – Myclass. Myattribute**

Syntax - class classname

statement – l

:

#statement - N

-Objects - Object is an entity that has a state and behavior associated with it. it may be any real world object like a mouse, keyboard, chair, table, etc. integers, strings, floating – point numbers, even arrays, and dictionary, are all object.

An object consists of –

- **State** – it is represented by the attribute of an object. It also reflect the properties of an object.
- **Behavior** – it is represented by the method of an object. It also reflect the response of an object to other objects.
- **Identify** – it gives a unique name to an object an enables one object to interact with other objects.

Obj = dog ()

Before diving deep into object and classes let us understand some basic keyword that will we used while working with objects and classes.

1> The self

- Class method must have an extra first parameter in the method definition. We do not give a value for this parameter value for this parameters when we call the method, python provides it
- If we have a method that's takes no arguments , then we still have to have one argument.

- This is similar to this pointer in c ++

When we call a method of this object as my object.
Method (arg 1, arg 2), this is automatically converted by python
into my a class. Method (my object , arg 1, arg 2).

2> The _init_ method _

It is similar to constructions in c ++ and java. It is run
as soon as an object of a class is instantiated. The method
is useful to do any initialization you want to do with your
objects.

-encapsulation in python -

It is a mechanism of wrapping the data (variable)
and code acting on the data (methods) together as a single unit.
In this, the variables of a class is hidden from other classes, and
can be accessed only through the methods of there current class.

Class base : # base class

Def _init_ (self):

Self _a = 2 # protected member

Class derived (base): # derived class

Def _init_ (self) :

calling constructor of base class

Base _init_ (self)

print ("calling protected member of base class Self _a)

modify the protected variable :

Self _a = 3

Print (“calling modified protected member outside

class :”, self._ a)

obj 1 = derived ()

obj 2 = base ()

print (“accessing protected member of obj 1 :”, obj 1. – a)

print (“accessing protected member of obj 2 :”, obj 2. – a)

-class instance attribute –

Class attribute belong to the class itself they will be shared by all the instance. Such attributes are defined in the class body parts usually at the top, for legibility.

Class sample class :

Count = 0

Def increase (self):

Sample class. Count + = 1

Si. = sample class ()

Si. Increase()

Print (si. Count)

S2 = sample class ()

S2. Increase ()

o/p – 1

Print (s2 count)

2

Print sample class. Count)

3

Unlike class attributes, instance attribute are not shared by object. Every object has list own copy of the instance attribute.

To list the attributes of an instance/object , we have two function-

- 1. Vars()- Jhis function is displays the attribute of an instance in the from of dicionary.**
- 2. dir()- This function displays more attributes then vars functions, as it not limited to instance. It displays the class attributes as well . It also displays the attributes of its ancestor classes.**
- 3.**

➤ Class member access-

Attribute of a class are function object that defines corresponding method of its instance . They are used to implement access controls of the classes.

Built -in functions-

- 1. getattr()- used to access the attribute of object.**
- 2. hasattr()- used to check if an attribute exist or not.**
- 3. setattr()- used to set an attribute.**
- 4. delattr()- used to delete an attribute.**

➤ Class method vs static method-

Class method is a method that is bound to the class and not the object of the class.

They have the access to the state of the class as it takes a class parameter that to the class and not the object instance.

It can modify a class state that would apply across variable that will be applicable to all the instances.

Static method doesn't receive an implicit first argument.

Syntax- class c (object):

@staticmethod

Def fun (arg1, arg2,.....):

returns : a static method for function fun.

A static method is also a method is also a method that is bound to the class and not the object of the class.

A static method can't access or modify the class state. It is present in a class because it makes sense for the method to be present in class.

-class method vs static method-

1. A class method takes cls as the first parameter while a static method needs no specific parameters.

2. A class method can access or modify the class state while a static method can't access or modify it.

3. In general static methods know nothing about the class state. They are utility-type method that take some

Parameters and work upon those parameters. On the other hand class methods must have class as a parameter.

- **Inheritance –**

It is the capability of one class to derive or inherit the properties from another class.

Advantage –

- it represents real – world relationship well.**
- it provide reusuability of a code. We don't have to write the same code again and again . also, it allows us to add more features to a class with out modifying it.**
- it is transitive in nature, which means that if class B inherits from another class A, then all the subclasses of B would automatically inherit from class A.**

Class person (object):

constructor

Def_ init_ (self, name):

Self. Name = name

to get name

Def get name (self):

Return self.name

to check if this person is an employee

Def is employee (self):

Return false

inherited or sub class

Class employee (person):

here we returns true

Def is employee (self):

Returns true

```
# driver code
emp = person ("geek 1")
print (emp. Get name ( ), emp. Is employee ( ) )
emp = employee ("geek 2")
print (emp. Get name ( ), emp. Is employee ( ) )
```

o/p - ('geek 1', false)

('geek 2', true)

-what is object class ?

Like java object class, in python , object is a root of all classes.

Example – class subclass_ name (super class _ name):

Class person (object):

Def _ init_ (self, name, id number):

Self. Name = name

Self. Id number = id number

Def display (self):

print (self. Id number)

child class

Class employee (person):

Def _ init_ (self, name, id number, salary, post):

Self. Post = post

Person. – init_ (self, name, id number)

A = employee ('Rahul', 8853, 200000, "intern")

a. display ()

-Different forms of Inheritance-

1. **Single Inheritance**- when a child class inherits from only one parent class , it is called single inheritance.

2. **Multiple Inheritance**- when a child class inherits from multiple parent classes , it is called multiple inheritance.

Class best(object):

Def- init -(self):

Self. Str1= "geek1"

print ("Base1")

Class Base2 (object):

def-init- (self):

Self.Str2= "geek2"

print ("Base2")

Class Deriver (Based1, Base2);

def-init-(Self):

Calling constructor of Base1 and Base2 classes

Bese1.-init-(self)

Bese2.- init-(self)

print("Derived")

def print strs (self):

print(self. str1, self.str2)

Ob= Derived()

Ob. Printstrs()

OP/ Base1	base 1
Base 2	base 2
Derived	Derived
	Geek 1 geek 2

3> multi- level inheritance –

when we have a child and grandchild relationship.

4> Hierarchical inheritance –

More than one derived classes are created from a single base.

5> Hybrid inheritance –

This form combines more than one from of inheritance . basically, it is a blend of more than one type of inheritance.

Private member of parent class –

We don't always want the instance variable of the parent class to be inherited by the child class i.e., we can make some of the instance variables of the parent class private, which wont be available to the child class.

• Polymorphism –

It means having many forms. In programming

Polymorphism means the same function name (but different signature) being used for different types.

Example - **inbuilt polymorphism function –**

```
print (len (“geeks”))
```

```
print (len ([10, 20, 30]))
```

o/p - 5

Example of user – defined polymorphism function –

Def add (x, y, z = 0):

Return x + y + z

print (add (2, 3))

print ((2, 3, 4))

o/p – 5

9

-polymorphism with class method –

Class India ():

Def capital (self):

print (“new Delhi is a capital of India”)

Def language (self):

**print (“Hindi is the most widely spoken
language in India”)**

def type (self):

print (“India is a developing country”)

class USA ():

def capital (self):

print (“washithgton, d. c. is the capital of USA”)

def type (self):

print (“USA is a developed country”)

obj_ Ind = India ()

Obj_ USA = USA ()

For country in (obj_ Ind, obj_ USA):

Country. Capital ()

Country. Language ()

Country. Type ()

➤ Polymorphism with inheritance –

Polymorphism lets us define methods in the child class that have the same name as the methods in the parent class. In inheritance, the child class inherits the method from the parent class. However, it is possible to modify a method in a child class that it has inherited from the parent class.

This is particularly useful in cases where the method inherited from the parent class doesn't quite fit the child class. In such cases, we re_ implement the method in the child class. This process of re_ implementing a method in the child class is known as method overriding.

```
Class bird :  
    Def intro (self):  
        print ("there are many types of birds")  
    Def flight (self):  
        print ("most of the bird can fly but some  
cannot")
```

```
Class sparrow (bird):  
    Def flight (self):  
        print ("sparrows can fly")  
Class ostrich (bird):  
    Def flight (self):  
        print ("ostriches cannot fly")  
Obj_ bird = bird ()  
Obj_ spr = sparrow ()  
Obj_ ost = ostrich ()  
Obj_ bird = intro ()  
Obj_ bird = flight ()
```

Obj_ spr. Intro ()
Obj_ spr. Flight ()
Obj_ ost. Intro ()
Obj_ ost. Flight ()

- **Abstract class –**

An abstract class can be considered as a blue print for other classes. It allows you to create a set of methods that must be created within any child classes built from an abstract class. A class which contains one or more abstract methods is a class that has a declaration but does not have an implementation. While we are designing large functional units we use an abstract class.

-by defining an abstract base class, you can define a common application program interface (API) for a set of sub classes.

Example – class polygon (ABC):

@ abstract method

Def noof sides (self):

Pass

Class tringle (polygon):

overriding abstract method

Def noof sides (self):

print (“I have 3 sides”)

Class Pentagon (Polygon):

Overriding Abstract Method

Def Noof Sides (Self):

print (“I Have 5 Sides”)

Class Hexagon (Polygon):

Overriding Abstract Method

Def Noof Sides (Self):

print (“I Have 6 Sides”)

Driver Code

R = Triangle ()

R. = Noof Sides ()

K = Quadrilateral ()

K. = Noof Sides ()

R = Pentagon ()

R = Noof Sides ()

K = Hexagon ()

K = Noof Sides ()

O/P – I have 3 Sides

I have 4 Sides

I have 5 Sides

I have 6 Sides

Thank You Message

Thank you for choosing "Python Programming for Beginners." I truly appreciate the time and effort you've invested in reading this ebook. It was created with the goal of making Python programming accessible, enjoyable, and practical for beginners like you.

Learning to code is an exciting journey, and I hope this book has provided you with a solid foundation and the confidence to continue exploring the world of programming. Whether you're pursuing Python for personal development, a career change, or as a hobby, remember that every step forward, no matter how small, is progress.

I encourage you to stay curious, keep experimenting with new ideas, and never hesitate to ask questions or seek help. The tech community is vast and supportive, and I am always excited to hear how others are applying what they've learned.

If you found this ebook helpful, I'd be grateful if you could share your feedback or recommend it to others who are starting their programming journey. Your support means a lot and helps me create even more valuable content for readers like you.

Once again, thank you for being a part of this journey, and I wish you the very best in your Python programming endeavors!

Happy coding!

Warm regards,

Mr_Circuits (Subrata Sasmal)

The background features a large, stylized Python logo. The top half of the logo is light blue and the bottom half is light yellow. A thin horizontal line passes through the center of the logo, behind the text.

HAPPY CODING

By Mr_circuits